COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 \| Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | October 20, 2026 | **DATE SUBMITTED:** | October 20, 2026 |
| **NAME:** | Niña Denise M. Santiago<br>Troy Jerron A. Bautista | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**HANDS - ON ACTIVITY NO. __14.1__** **Algorithm Complexity**

**1. PROCEDURE OUTPUT**

| | Best Case | Worst Case | Analysis |
|---|---|---|---|
| **Algorithm 1** | $O(n^2)$ | $O(n^2)$ | Selection Sort has the same time complexity in both cases because it must scan the remaining unsorted elements to find the minimum value, even if the array is already sorted. |
| **Algorithm 2** | $O(n \log n)$ | $O(n^2)$ | Quick Sort is efficient when the pivot divides the array into nearly equal parts, that's the Best Case. Its worst case happens when the partitions are highly unbalanced, such as repeatedly choosing the smallest or largest element as the pivot. |

*Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools*

| | Algorithm 1 | Algorithm 2 |
|---|---|---|
| | Selection Sort vs. Quick Sort<br>Input Size: 1000 | |
| **Best Case** | Selection Sort Best Case: 262.300 ms | Quick Sort Best Case: 49.000 ms |
| **Worst Case** | Selection Sort Worst Case: 370.300 ms | Quick Sort Worst Case: 771.200 ms |

*Table 14-2. Best- and Worst-Case Analysis using Experimental Tools*

**Adjusted Codes from HOA 7.1 (Selection Sort)**

```cpp
template <typename T>
int routine_smallest(T arr[], int k, const int maxSize) {
    int position = k;
    T smallestElement = arr[k];

    for (int j = k + 1; j < maxSize; j++) {
        if (arr[j] < smallestElement) {
            smallestElement = arr[j];
            position = j;
        }
    }

    return position;
}

template <typename T>
void selectionSort(T arr[], int maxSize) {
    for (int i = 0; i < maxSize - 1; i++) {
        int minVal = routine_smallest(arr, i, maxSize);
        std::swap(arr[i], arr[minVal]);
    }
}
```

**Adjusted Codes from HOA 8.1 (Quick Sort)**

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;

            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

*Table 14-2-1. Codes For Experimental Analysis*

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

| Input Size (n) | Elapsed Time for Binary Search (Worst-Case) | Elapsed Time for Linear Search (Worst-Case) |
|---|---|---|
| 1,000 | Linear Search vs. Binary Search<br>Input Size: 1000 | |
| | Binary Search Worst Case: 0.100 ms | Linear Search Worst Case: 0.700 ms |
| 10,000 | Linear Search vs. Binary Search<br>Input Size: 10000 | |
| | Binary Search Worst Case: 0.200 ms | Linear Search Worst Case: 5.300 ms |
| 100,000 | Linear Search vs. Binary Search<br>Input Size: 100000 | |
| | Binary Search Worst Case: 0.300 ms | Linear Search Worst Case: 58.300 ms |

| Analysis |
|---|
| The results show that Binary Search is faster than Linear Search in all input sizes. For 1,000 elements, Binary Search took 0.100 ms, while Linear Search took 0.700 ms. When the input increased to 10,000 elements, Binary Search took only 0.200 ms, while Linear Search increased to 5.300 ms. For 100,000 elements, Binary Search took 0.300 ms, while Linear Search reached 58.300 ms.<br><br>This shows that Binary Search is more efficient for large amounts of data because it repeatedly divides the search area in half.While on the other hand, Linear Search checks each element one by one until it finds the target. Overall, the experimental results support the theoretical time complexity: Binary Search has a worst-case complexity of $O(log\ n)$, while Linear Search has $O(n)$. Therefore, Binary Search is a better choice when searching large, sorted datasets. |

Theoretical Analysis

| Pseudocode (Linear Search) | Pseudocode (BinarySearch) |
|---|---|
| Main Program<br>START<br>SET max_size to 1000, 10000, or 100000<br>FOR I from 0 to size<br>  arr[i] = i + 1<br>ENDFOR<br>CALL displayArr with A<br>CALL linearSearchArr with arr, max_size, and target<br>END<br><br>Function for linearSearchArr<br>FUNCTION linearSearchArr (T arr[], int size, T target)<br>  FOR I from 0 to size<br>  IF arr[i] == target THEN<br>  RETURN I<br>  RETURN -1<br>ENDFUNCTION | Main Program<br>START<br>SET size to 1000, 10000, or 100000<br>FOR I from 0 to size<br>  arr[i] = i + 1<br>ENDFOR<br>CALL displayArr with A<br>CALL binarySearchAR with arr, max_size, and target<br>END<br><br>Function for binarySearchAR<br>FUNCTION binarySearchAR (int arrSize, T dataFind, T list[])<br>  SET low = 0<br>  SET high = arrSize - 1<br>  WHILE low <= high<br>  mid = int((low + up)/2)<br>  IF arr[mid] == target<br>  RETURN mid<br>  ELSE IF arr[mid] < target<br>  low = mid + 1<br>  ELSE<br>  high = mid - 1<br>  RETURN -1<br>ENDFUNCTION |
| **Time Complexity of Linear Search:** 3n + 3 | **Time Complexity of Binary Search:** $T(n) = T(n/2) + c$ |
| **Space Complexity of Linear Search:** 4n + 16 bytes | **Space Complexity of Binary Search:** 4n + 28 bytes |

| Conclusion (Santiago) |
| --- |
| In this activity, I learned that analyzing the time and space complexity of algorithms helps us understand how efficiently a program works when handling different amounts of data.<br><br>In the sorting activity, I learned that Selection Sort has a time complexity of $O(n^2)$ in both the best and worst cases because it still needs to check the remaining elements to find the smallest value. Meanwhile, Quick Sort has a best-case time complexity of $O(n \log n)$ when the pivot divides the data into nearly equal parts, but its worst-case complexity is $O(n^2)$ when the partitions are unbalanced.<br><br>In the supplementary activity, I observed that Binary Search was faster than Linear Search in all three input sizes. For 1,000 elements, Binary Search took 0.100 ms, while Linear Search took 0.700 ms. For 10,000 elements, their running times were 0.200 ms and 5.300 ms, respectively. For 100,000 elements, Binary Search took only 0.300 ms, while Linear Search took 58.300 ms. These results show that Binary Search is more efficient for large amounts of sorted data because it divides the search area in half, while Linear Search checks elements one by one.<br><br>Overall, this activity taught me the importance of comparing theoretical and experimental results to understand algorithm performance. I realized that choosing the right algorithm can reduce running time and make programs more efficient, especially when working with large amounts of data. |

| Conclusion (Bautista) |
| --- |
| In conclusion, complexity measures time and space of an algorithm to determine its efficiency. This helps determine what algorithm can run faster and smoother without taking too much time and not taking up too much space.<br><br>In the sorting activity, I learned that selection sort has a best case and worst case of O(n^2) because the algorithm uses a nested loop in order to find the minimum value. Quick sort has a best case of O(n log n) when the array is divided into nearly equal parts, while its worst case is O(n^2) when the partitions are very unbalanced.<br><br>In the supplementary activity, we compared linear search to binary search and found out that binary search is |

faster than linear search. In all the runs with input sizes of 1000, 10000, and 100000, it shows that binary search ran faster than the linear search and also showed as the input sizes gets larger, the linear search's runtime gets bigger.

Overall, theoretical and experimental analysis is very important to understand how efficient the algorithm performs. Using these analyses, we can find out which program or algorithm can reduce runtime and how it handles large amounts of data.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **ChatGPT**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Debugging Assistance and Random Set of Data Generation**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **ChatGPT helped with challenging debugging items that includes (challenges with vs code setups, code typo corrections, etc.) and Generation of random set of data for consistency, it did not replace any original thinking, analysis, authorship, or completion of the required work.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.